Fast intro to R

Germain Poullot

April 2026

Contents

Where to find good courses on R :
- Kristina Schubert lecture notes: https://www.kristina-schubert.de/statistik-mit-r/
- R for Data Science: https://r4ds.hadley.nz/
- Harvard YouTube video (9h): https://www.youtube.com/watch?v=g_3IKHG-rfA

Where to start seaching for good data:
- In German https://www.destatis.de/DE/Home/_inhalt.html
- In French https://www.insee.fr/fr/accueil
- Worldwide https://en.wikipedia.org/wiki/List_of_national_and_international_statistical_services

Whenever you see a "**Q**:" in this document, it stands for "**question**:", and it means you should grab your computer (or pen and paper) in order to workout the solution.

1 Course 1: First steps, the vector-table mindset, descriptive statistics

1.1 Downloading R

Go to https://www.r-project.org/, and download the latest (stable) version of R. Install it.
Go to https://posit.co/downloads and download the latest (stable) version of RStudio. Install it.

1.2 Making my first table

1.2.1 Where am I working?

Open RStudio; you should see 4 sub-windows.

First, we need to decide where we will run our code. Currently, R is running wherever you initially asked it to run, but you probably forgot where it is, so let's ask:

```
getwd()
```

If you want to change it (which you probably do, because R needs to handle a lot of data at once, so you would prefer to make it run in its own folder):

```
setwd("C:/the_path_to_my_directory")
```

`wd` stands for "working directory". You can do things more easily with RStudio: in the top-right, click on the RStudio logo, and open a new directory with the name "Course1".

1.2.2 Can you do math?

Now, let's check that R knows math. In the console (bottom left), write

```
5+3
```

Then execute it.

Also check the result of $3 + 5$.

Always writing in the console will be very annoying, so let's open a new *script* (i.e. a text file that contains the commands to execute): either go to File > New File > R Script, or press Ctrl+Shift+N.

Write again $5 + 3$ on the first line of your script, and press Ctrl+Enter.

This executes your current line. If you want to execute everything from the beginning, press Ctrl+Shift+Enter.

1.2.3 Vectors, tables

R uses *vectors* and *tables* (a.k.a. *frames*).

To create a vector, write (`c` stands for "combine"):

```
c (2, 5, −2, 2, −1)
```

I want to save this vector in a variable with a meaningful name.

```
Temp = c (2, 5, −2, 2, −1)
# or
Temp <− c (2, 5, −2, 2, −1)
```

These are the temperatures for a week (at the beginning of spring). Hence, let's create the days of the week.

```
Week = c ("Monday", 'Tuedsday', 'Wednesday', 'Thursday', 'Friday',
"Saturday", 'Sunday')
```

Note that, in the top-right, you have the names of your variables and a summary of what they contain.

It is very annoying to have these two vectors separated; I want them in a common *table*.

```
my_table = data.frame(Week, Temp)
```

Problem: I have 5 temperatures for 7 days of the week, so I cannot make a table. Add 0 and -1 to the temperatures. Re-execute `my_table = ...`.

Let's view my table:

```
View(my_table)
```

Note that the names of the columns were automatically set to be the names of your variables. You can access the columns of a table using the $ sign, e.g. `my_table$Temp` is the column containing the temperatures.

Let's compute the number of different temperatures, their mean, and their variance.

```
unique(my_table$Temp)
length(unique(my_table$Temp))

mean(my_table$Temp)
var(my_table$Temp)
```

To access the data inside my table, I can call them directly (remember: in matrices, the first entry is the row number, the second is the column number). The numbering starts at 1.

```
my_table[1, 2]
my_table[2, 1]

my_table[3:4, 1:2]
my_table[c(1, 3), 2]

my_table[1, ]
my_table[5, ]
my_table[, 1]
my_table[, 2]

my_table$Week[3]
my_table$Week[1:4]

my_table[9]
```

1.2.4 Filters

R is very good at applying the same operation to a whole vector. I want to know how many days had a negative temperature.

```
my_table$Temp < 0
my_table$Temp <= 0
```

This gives you a vector of answers. Note that, unlike many languages, in R the words `TRUE` and `FALSE` are uppercase, which is (annoying but) important.

I want the part of my table that contains only the days with negative temperatures.

```
which(my_table$Temp < 0)
my_table[my_table$Temp < 0, ]
subset(my_table, Temp < 0)
```

I want the mean temperature of the negative days. Try:

```
mean(my_table[my_table$Temp < 0, ])
```

It doesn't work: `my_table[my_table$Temp < 0, ]$Temp` is still a table. You didn't specify which column you want the mean of. Let's correct that:

```
mean(my_table[my_table$Temp < 0, ]$Temp)
```

1.2.5 Coordinate-wise operations

I want to estimate how much I will pay for buying my fruits. Let's download some data on fruit prices (because I'm too lazy to invent some myself).

```
url = "https://gist.githubusercontent.com/MamathaReddygari/
0b52821de0d7d92928c00e648f67f00c/raw/
70e3c578bbf49755e99e2af528c20be11d47a3d5/fruits.csv"

retail = read.csv(url)
View(retail)
```

A `csv` file is a file with *comma-separated values*. You can open it with a text editor, but when it becomes very large, you can open it directly in R.

I am not buying every possible fruit, so let's list the ones I want and extract them from the data.

```
my_fruits = c ("Bananas", "Apples", "Pears", "Figs", "Blueberries")

my_groceries = subset(retail, Fruit %in% my_fruits)
View(my_groceries)
```

I don't like frozen fruit, so let's remove it. The symbol `!` in front of a boolean statement means "not".

```
my_groceries = subset(my_groceries, !Form == "Frozen")
```

Similarly, I only care about the name of the fruit, the form, and the price.

```
my_groceries = my_groceries[, c("Fruit", "Form", "RetailPrice")]
```

As I am a bit of a perfectionist, I don't like that the rows of my table currently have strange numbers (they are the numbers from the original table `retail`). To fix this, there is an easy way: just remove the row names and let R recreate them from scratch.

```
rownames(my_groceries) = NULL
```

I go to the supermarket: I buy 200g of bananas, 500g of apples, 100g of pears, 150g of figs, and 120g of blueberries. Let's add this to my table.

```
my_groceries$Quantities = c(0.2, 0.5, 0.12, 0.15, 0.1)
```

Now, let's compute the price I paid for each item. Remember that R is very good at performing coordinate-wise operations on vectors.

```
my_groceries$PricePaid = my_groceries$RetailPrice * my_groceries$Quantities
sum(my_groceries$PricePaid)
```

1.3 Final exercise of the first session

1.3.1 Installing a package

The next table I want to work with is a common example already available in R, but we need to download a package to access it.

```
# This takes time, but you need to do it only once:
install.packages("nycflights13")

# This you need to redo each time:
library(nycflights13)
```

This package contains (among other things) a variable `flights`. Write the following and read in the bottom-right panel to understand what this variable contains:

```
?flights
```

Q: How many rows and columns does this table contain?
Q: List the years appearing in the table. How many different departure times are there?

1.3.2 Means and standard deviation depending on another column

Q: How many flights were there in March? What is the mean departure delay?
Q: Create a table `DELAYS` with one column named `month`.
I want to add a column computing the number of flights for each month. Let's use the `for` syntax.

```
for (i in c(1:12)) {
  DELAYS$nbr_flights[i] = sum(flights$month == i)
}
```

Q: Don't forget to view your table. Check that the total number of flights is indeed the sum of the number of flights for each month.
Now, let's add the mean departure delay and its standard deviation. This data is in the column `flights$dep_delay`. We will see one way now and a better way next time.

```
for (i in c(1:12)) {
  DELAYS$mean_delay[i] = mean(flights$dep_delay[flights$month == i])
  DELAYS$sd_delay[i] = sd(flights$dep_delay[flights$month == i])
}
```

All the means and standard deviations are `NA`. Why?
Let's debug a few things. Execute `flights$month == 7` to see the result, then `flights$dep_delay[flights$month == 7]`. How many values are there in this vector? Do you see some `NA` values?
The word `NA` means *not available*. R requires you to decide how to handle missing data, so it will never decide on its own what to do. Here, we want to remove these `NA` values (which is very different from considering them equal to 0). Luckily, there is a straightforward way to do this in the `mean` and `sd` functions (the letters `rm` mean "remove"):

```
mean(flights$dep_delay[flights$month == 7], na.rm=TRUE)
```

Q: Now, correct your code to compute the mean and standard deviation of the departure delays.

2 Course 2: `tidyverse`, making drawings, intro to linear regression

2.1 Downloading `tidyverse`

The package **`tidyverse`** ("tidy" = sorted in a clean way; "-verse" stands for universe) is efficient for cleaning and visualizing data. First of all, let's install it and make it available in our session. Create a new directory **`SecondCourse`**, then write:

```
install.packages("tidyverse") # Takes time, may complain

library(tidyverse)
```

To discover what the package can do, we will work on the **`tibble`** called **`storms`**, in order to extract data on hurricanes. A **`tibble`** is just a variant of a **`table`**, but created by **`tidyverse`**.

Visualize the content of the **`tibble`** named **`storms`**.

`tidyverse` is not just one package; it actually contains many sub-packages with their own names: we will mainly use **`dplyr`** and **`ggplot2`** today.

2.2 Cleaning data

2.2.1 Selecting columns and rows, using piping

To call a function from a package, you can use double colons, e.g. **`dplyr::select`**. However, **`tidyverse`** loads many functions directly, which means you can just write **`select`** instead. Type:

```
?select
?dplyr::select
```

Then understand what the following does:

```
select(storms,
       -c(lat, long)
       )

select(storms,
       -c(lat, long, ends_with("diameter"))
)

filter(storms,
       status == "hurricane")
```

Typically, in R, you want to perform an operation on a table (or **`tibble`**), then perform another action on the result, and so on. Writing $f(g(h(\dots)))$ quickly becomes annoying, so R provides a convenient alternative called *piping*, denoted by **`|>`** (like a smaller version of $\mapsto$). This operator passes what precedes it as the first argument of the following function. Look at the following **`tibble`**.

```
storms |>
  select(-c(lat, long, ends_with("diameter"))) |>
  filter(status == "hurricane")
```

Save the resulting **`tibble`** in a variable named **`hurricanes`**, view it, compute the number of different hurricane names, and the mean pressure.

We now want to sort the data efficiently. To this end, we will use another function from **`tidyverse`** that arranges the **`tibble`** in ascending (or descending, if specified) order for a given column.

```
arrange(hurricanes, wind)
arrange(hurricanes, desc(wind))
```

```
arrange(my_tibble, desc(wind), name)

arrange(hurricanes, desc(wind), name) |>
  distinct(name, year)
```

The function `distinct` allows you to keep only the rows with unique values.

Now that we are almost satisfied with the data extracted from the original `tibble` named `storms`, we can save it as a `.csv` file.

```
clean_hurricanes = arrange(hurricanes, desc(wind), name) |>
  distinct(name, year, .keep_all = TRUE)

clean_hurricanes |>
  select(c(year, name, wind, pressure)) |>
  write.csv("hurricanes.csv", row.names = FALSE)
```

2.2.2 Grouping, slicing, summarizing

Open a new script, say `SecondScript.R`, and load the `tibble` named `hurricanes` that you have just saved.

Now, we can do what we did at the end of the last session, but faster and more cleanly. First, we group the content of the table by year.

```
hurricanes |>
  group_by(year)
```

The content is not only visually grouped by year; it is also interpreted by R as a collection of "big rows" named 1980, 2019, 1988, 2005, etc., each containing sub-rows. Hence, we can perform operations on this structure. For instance, let's remove all rows except the first one: if we do this on a normal `tibble`, we keep only one row, but here we keep one row per group.

```
hurricanes |>
  group_by(year) |>
  slice_head()

hurricanes |>
  group_by(year) |>
  slice_min(order_by = pressure) |>
  filter(year >= 1980 & year >= 2000)
```

We can also create summaries of the groups, e.g. count the number of elements per group or compute the mean of some value:

```
hurricanes |>
  group_by(year) |>
  summarise(n())

hurricanes |>
  group_by(year) |>
  summarise(number_of_hurricanes = n())
```

Q: Construct a `tibble` with 3 columns indicating the mean wind speed and the number of hurricanes for each year.

2.2.3 Fixing poorly formatted data

Q: Download the `tibble` named `students` and understand the problem.
As this problem is quite common, there is a direct solution in the `tidyverse`:

```
students = pivot_wider(students,
             id_cols = name,
             names_from = attribute,
             values_from = value
             )
```

Q: Try to create a `tibble` containing the mean GPA (Grade Point Average) in each major.
Q: Notice that you have a problem, identify it, and correct it (you can use `as.numeric(...)` for instance).
Q: Type `?pivot_longer` and strengthen your understanding.
As you can imagine, there are many other problems that can occur when working with data produced by others. You now know the fundamentals: check whether `tidyverse` can handle your problem. In the future, you can use the `tidyverse` documentation, search the Internet, or ask an AI which commands to use; but you should not reinvent the wheel!

2.3 Final exercise on cleaning data: cleaning strings

Q: Download `favorite_subjects.csv` into a `tibble` named `best`.
Q: Group your data to count how many times each subject has been voted for, and sort them in decreasing order.
Q: Notice that there are several problems. Visualize the beginning of your `tibble` to understand what needs to be done.
To remove spaces at the beginning and end (respectively inside) of a string, you can use `str_trim()` (respectively `str_squish()`). To manipulate case (upper/lower), you can use `str_to_title()`, `str_to_upper()`, or `str_to_lower()`.
Q: Start cleaning your data, and look at your poll again.
You still have a German versus English issue. This is more delicate, so let's handle it manually.
Q: Create the following two vectors, and use a clever `for` loop to replace German words with English ones in `best` (use the next two commands and understand what they do). Then compile your poll and save the result in a variable named `poll`.

```
German = c('Statistik', 'Mathematik', 'Informatik')
English = c('Statistics', 'Mathematics', 'Computer Science')
```

```
any(best$Subject %in% German)
all(best$Subject %in% English)
```

Q: Think about (do not implement it) a way to combine the results for algebra, statistics, and mathematics, and those for data science and computer science.

2.4 Making nice drawings

2.4.1 The basics of `ggplot`

To create a drawing, R proceeds step by step. The first step is to create a blank page with `ggplot(...)`. We are going to visualize the recent `poll` that you just made, so let's pass this variable to our plot: `ggplot(poll)`. Nothing happens, because R (or more precisely `tidyverse`) does not yet know what you want to plot exactly, but the data are loaded.
Second, we need to specify what goes on the x-axis and what goes on the y-axis. This is done using *aesthetics*, which is a somewhat convoluted concept: each function that we are going to use is supposed to do something on its own, and you can pass it some aesthetics to explain how it is

supposed to do its job. Hence, we ask `ggplot` to load the data from `poll`, and we specify that the x-axis should come from the vector `poll$Subject` while the y-axis should come from `poll$votes`. As `poll` is already loaded, you only need to write:

```
ggplot(poll, aes(x = Subject, y = votes))
```

Err... that's interesting, but where is my plot? Well, you did not tell R what you wanted to plot: you have set up the scenery, but not the play. The "play" here is called a *geometry*. As we are doing a poll, the geometry that we want to use is probably a histogram, that is to say `geom_col()`. You can add it (literally) to your plot, as follows:

```
ggplot(poll, aes(x = Subject, y = votes)) +
  geom_col()

ggplot(poll, aes(y = Subject, x = votes)) +
  geom_col()

ggplot(poll, aes(x = Subject, y = votes)) +
  geom_col() + coord_flip()
```

Note that, in the last call, we have added `coord_flip()`, which flips the x-axis and the y-axis. This is a basic example of how plotting works: you create a plot, you add the geometry, then you add a bunch of modifiers on top. Everything is done with addition (which is commutative, so it does not depend on the order[1]).

You have probably noticed that the column plot has reordered your data alphabetically. This is annoying, so we bypass that. We also add some labels to our plot.

```
ggplot(poll, aes(x = reorder(Subject, votes, decreasing = TRUE), y = votes)) +
  geom_col() +
  labs(x = "Subject", y = "nb-votes", title = "Favorite-subjects")
```

The following are discussions of some common aesthetics and modifiers that you may want to apply to your plot. You should try them at least once: even though it may be a bit boring, it will make you more comfortable with plotting. There are many more options, and many variations of each. I leave it to you to imagine the possibilities[2].

Changing the limits of your scale

```
ggplot(poll, aes(x = Subject, y = votes)) +
  geom_col() +
  scale_y_continuous(limits = c(0, 40))
```

Filling the columns with colors

```
ggplot(poll, aes(x = reorder(Subject, votes, decreasing = TRUE), y = votes)) +
  geom_col(aes(fill = Subject))
  labs(x = "Subject", y = "nb-votes", title = "Favorite-subjects")
```

Adapting for colorblind people (Viridis is the name of a color standard in R , and `d` stands for "discrete")

```
ggplot(poll, aes(x = reorder(Subject, votes, decreasing = TRUE), y = votes)) +
  geom_col(aes(fill = Subject)) +
  scale_fill_viridis_d("Theme") +
  labs(x = "Subject", y = "nb-votes", title = "Favorite-subjects")
```

[1] But the human being reading your code might care deeply about the order...

[2] Note that R accepts both British and American spelling for words like `color`/`colour`.

Removing the legend

```
ggplot(poll, aes(x = reorder(Subject, votes, decreasing = TRUE), y = votes)) +
  geom_col(aes(fill = Subject), show.legend = FALSE) +
  scale_fill_viridis_d("Theme") +
  labs(x = "Subject", y = "nb votes", title = "Favorite subjects") +
  theme_classic()
```

By now, you should be satisfied with your first plot, so we can save it. Remember that a plot is an object that exists in your computer's memory, so you can store it in a variable. The unit `px` stands for "pixel": the default unit is inches (≈ 2.54 cm), which is not convenient in most cases.

```
p = ggplot(poll, aes(x = reorder(Subject, votes, decreasing = TRUE), y = votes)) +
  geom_col(aes(fill = Subject), show.legend = FALSE) +
  scale_fill_viridis_d("Theme") +
  labs(x = "Subject", y = "nb votes", title = "Favorite subjects") +
  theme_classic()

ggsave("FavoriteSubject.png", plot = p, height = 900, width = 2400, units = "px")
```

2.5 Final exercise on plotting: drawing the Saffir–Simpson scale

We want to know whether there is some dependence between wind speed and pressure in hurricanes. (If you have unloaded the variable `hurricanes`, remember that you saved it in a `.csv` file, so you can reload it at any time.)

Q: Make a plot of the `hurricanes` variable, with pressure on the x-axis and wind speed on the y-axis, using points (not columns). The theme should remain classic, and the title of your plot should be "Hurricanes".

Q: Set the shape of the points to `shape = 1`, and color them according to the year the hurricane occurred.

Q: It is hard to distinguish between the points; can you jitter them (use `geom_jitter`)? Now that you can see better, what kind of hurricanes would you like to observe to complete your data?

Q: (Go back to the non-jittered point plot.) We want to categorize hurricanes. By definition, a hurricane is a storm with a wind speed greater than 64 knots (1 knot ≈ 1.852 km/h, so roughly 118 km/h), but according to the Saffir–Simpson scale, we can subdivide hurricanes further into categories depending on their wind speed:

- between 64 knots and 82 knots: category I;
- between 83 knots and 95 knots: category II;
- between 96 knots and 112 knots: category III;
- between 113 knots and 136 knots: category IV;
- beyond 137 knots: category V;

Using the following vector and the function `findInterval(x, vec)`, which finds between which values of the vector `vec` the values in `x` lie, add a column `category` to your `tibble` named `hurricanes`.

```
SaffirSimpson = c(64, 83, 96, 113, 137)
```

Q: We can indicate in the aesthetics of our plot that our data should be treated as several subtables. Using the following beginning, draw a point plot (with shapes 1), called "Hurricanes", in the classic theme, without a legend.

```
p = ggplot(hurricanes, aes(x = pressure, y = wind, color = factor(category))) + ...
```

Q: I do not like the colors, so let's change them using the following vector. To do so, you need to scale the colors manually, in the same way as you scaled the limits of the y-axis.

```
my_colors = c('green', 'orange', 'red', 'purple', 'black')
```

Q: Do not forget to store your plot in a variable named `p`. I want to add horizontal lines to the plot to separate the categories. Remember that we are adding elements to our plot, so nothing prevents us from adding another geometry. The geometry for a horizontal line is `geom_hline(...)`, and you can pass it an argument `yintercept = ...` to indicate the y-value of the line. Draw 6 horizontal lines, colored according to `my_colors`, at the heights given in the Saffir–Simpson scale (add 0.5 for cleaner separation). Use `linetype = 3` to obtain dotted lines.

2.6 Towards linear regression

As we have factored our data, not only can we treat their colors separately, but we can also apply other methods separately. For instance, let's add the line obtained by linear regression for each category of hurricanes (`lm` stands for "linear model"; smooth geometries are tools for computing smooth trends, and by default they use LOESS[3]).

Try to understand what `se` could mean.

```
p + geom_smooth(method = "lm", se = FALSE, show.legend = FALSE)
p + geom_smooth(method = "lm", se = TRUE, show.legend = FALSE)
```

Finally, for the sake of exploration (and because we may not have time to cover it in detail), let's briefly look at linear regression:

```
model <- lm(wind ~ pressure, data = hurricanes)
summary(model)
```

To get the coefficients, you can write:

```
model$coefficients[1]
model$coefficients[2]
```

If you want the *coefficient of determination* (usually denoted R^2), you can access it via:

```
r2 <- summary(model)$r.squared
r2
```

Finally, let me add the global regression and its coefficient of determination, instead of computing a regression for each category:

```
q = ggplot(hurricanes, aes(x = pressure, y = wind)) +
  geom_point(shape = 1, show.legend=FALSE) +
  labs(title = "Hurricanes") +
  theme_classic()
for (i in 1:6) {
  q = q + geom_hline(linetype = 3, yintercept = SaffirSimpson[i]+0.5, color=my_colors[i])
}
q + geom_smooth(method = "lm", se = TRUE, show.legend = FALSE) +
  annotate("text",
           x = Inf, y = Inf,
           label = paste0("R^2=", round(r2, 3)),
           hjust = 1.1, vjust = 1.5)
```

[3] Local polynomial regression

3 Course 3: Creating and sampling probability distributions

3.1 Visualizing usual distributions

Usual distributions are already implemented in R. Each implemented distribution comes in 4 ways (usually): *density*, *distribution*, *quantiles*, and *sampling*. For instance, for the uniform distribution, the built-in functions are `dunif`, `punif`, `qunif`, and `runif`. For disambiguation, say that $X \sim \text{Unif}([0,1])$, let f be the density function (i.e. the derivative of $t \mapsto \mathbb{P}(X \leq t)$), then we have:

- `dunif(t)` $= f(t)$
- `punif(t)` $= \mathbb{P}(X \leq t)$
- `qunif(z)` $= \min\{t \in \mathbb{R} \ ; \ \mathbb{P}(X \leq t) \geq z\}$ for $z \in [0,1]$
- `runif(n)` is a vector of length n where each entry is a sample from the uniform distribution

You can get more information using:

```
?dunif
?punif
?qunif
?runif
```

Let's make a table to visualize all this stuff.

```
x = (0:100) / 100
density = dunif(x)
distrib = punif(x)
quantile = qunif(x)

Unif = data.frame(x, density, distrib, quantile)
View(Unif) # It is weird to put quantile in this table...
```

Q: Explain why it is weird to put everything in the same table (the next example will be more striking).

We draw the density and the distribution using `ggplot`. **Nota Bene**: If you do not manage to install `tidyverse` (and `ggplot`), you can use `hist(...)` instead of `geom_histogram(...)`, and `curve(...)` instead of `stat_function(...)`. Good luck!

```
library(tidyverse)

ggplot(data.frame(x = c(-0.33,1.33)), aes(x = x)) +
  stat_function(fun = dunif, linewidth = 0.5, color='blue') +
  stat_function(fun = punif, linewidth = 0.1, color='red') +
  labs(title = "Density and distribution of Uniform(0,1)", y = "Density, Distribution") +
  ylim(0, 1.2) +
  theme_classic()
```

We do the same with the normal distribution, because we want to compare them. Comment on the change of `x`, and look at the `quantile` column in the `Norm` table.

```
?dnorm
?pnorm
?qnorm
?rnorm

x = (-500:500) / 100
density = dnorm(x)
distrib = pnorm(x)
quantile = qnorm(x)
```

```
Norm = data.frame(x, density, distrib, quantile)
View(Norm) # here, it is clear that quantile is a bad idea!

ggplot(data.frame(x = c(−5,5)), aes(x = x)) +
  stat_function(fun = dnorm, linewidth = 0.5, color='blue') +
  stat_function(fun = pnorm, linewidth = 0.1, color='red') +
  labs(title = "Density and distribution of Normal(0,1)", y = "Density, Distribution") +
  ylim(0, 1.1) +
  theme_classic()
```

We finish this introduction by comparing the quantile functions for the uniform and the normal distributions.

```
ggplot(data.frame(x = c(−0.1,1.1)), aes(x = x)) +
  stat_function(fun = qunif, color = 'green') +
  stat_function(fun = qnorm, color = 'orange') +
  ylim(−2.5, 2.5) +
  theme_classic()
```

3.2 Covariance matrix to analyze a shape via Monte Carlo method

Suppose you want to determine the shape of some plane object (formally fix $K \subseteq \mathbb{R}^2$), but you can only try asking whether a point is in the shape or not (formally, you have access to an oracle $f : (x, y) \mapsto \big((x, y) \in K\big)$). You know that the whole shape is contained in the square $[0, 100]^2$.

You can of course pick points regularly inside $[0, 100]^2$ and ask if they are in the shape (formally fix a large n and ask $f(x_i, y_j)$ for $x_i = \frac{100i}{n}$ and $y_j = \frac{100j}{n}$ for all $0 \leq i, j \leq n$), then draw the shape using a computer. However, this can be very costly, and it is usually impossible to do in real life (due to both measurement errors and experimental setups). One way around this is to use a Monte Carlo method: sample points at random in your shape. Sampling uniformly inside the shape directly is usually tricky[4], yet one can sample uniformly inside a larger shape (e.g. a disk or a square) and filter to keep only the points lying inside your shape.

The same principle applies if the shape is the result of a certain function: one can sample in the domain of definition, then apply the function to every sample point, obtaining a sample of the image shape. This sample will not necessarily be uniform enough, but there are ways to make it more uniform. We do not explore this aspect here, but focus on sampling uniformly in a square and then testing whether the points belong to the shape.

3.2.1 Sampling in my shape

First, let's sample uniformly inside a square.

```
XY = cbind(x = runif(100, ), y = runif(100))
View(XY)

nrow(XY)

cov(XY)
```

Q: What were you expecting the covariance matrix to be?

Then, we need to import a shape. I have made an auxiliary script in R that you need to download and save next to your current script, named `what_is_my_shape.R`. You can open it manually, but that would be cheating.

You can import the functions therein and test them. My shape is inside the box $[-1.5, +3] \times [-2, +2]$.

[4] For instance, I know no efficient algorithm to sample points uniformly at random in a polytope.

```
source("what_is_my_shape.R")

in_my_shape(4, 2)
in_my_shape(0, 0)
```

Now, we sample points uniformly at random inside $[-1.5, +3] \times [-2, +2]$, and visualize them.

```
XY = data.frame(x = runif(1000, min = -1.5, max = 3), y = runif(1000, min = -2, max = 2))
nrow(XY)

ggplot(XY, aes(x = x, y = y)) +
  geom_point()
```

We want to apply the function `in_my_shape` to every row of the table `XY`, but "sadly", I did not *vectorize* my function, that is to say I did not adapt it to use a vector (nor a data frame) as an argument, and return a vector (or a data frame). We can easily make do with that (`mapply` stands for "apply map to"):

```
mapply(in_my_shape, XY$x, XY$y)
```

This should be a vector of `TRUE` and `FALSE`, indicating which of the points you have randomly chosen are inside my shape, and which are not. We make a table out of that, and visualize it.

```
MonteCarlo = XY[mapply(in_my_shape, XY$x, XY$y), ]
View(MonteCarlo)

nrow(MonteCarlo)

ggplot(MonteCarlo, aes(x = x, y = y)) +
  geom_point()
```

Q: Without writing any new line, what would be your guess of the area of my shape? How can you make your guess more precise? (math question: does the method you are proposing converge? under which mode of convergence? how fast?)

3.2.2 Principal Component Analysis

My shape is weird, but it clearly has some fundamental axis. In the same way, clouds of points coming from real-life data usually have a fundamental axis. A very simple way to detect such a fundamental axis is called *Principal Component Analysis*, which means computing the eigenvector of the covariance matrix associated with the largest eigenvalue. Remember that the covariance matrix is a symmetric matrix, so all its eigenvalues are real, and the covariance matrix is diagonalizable.

We can directly compute the spectral decomposition of a matrix in R. The result is usually sorted in decreasing order of eigenvalues.

```
Sigma = cov(MonteCarlo)
eigen(Sigma)
```

Using `tidyverse`, we add arrows in the directions given by the eigenvectors, i.e. the columns of `eigen(Sigma)$vectors`. We scale each arrow according to its eigenvalue: hence their relative sizes indicate the importance of each direction compared to the other direction. If all the arrows have the same size, or if they are very small, then one should be doubtful about the existence of a fundamental axis: the point cloud is either very symmetric, or completely chaotic, or has many different axes/clusters, or the representation is not adapted (take a semi-log scale, or consider another transformation), or else.

```
ei = eigen(Sigma)$values
P = eigen(Sigma)$vectors

ggplot(MonteCarlo, aes(x = x, y = y)) +
  geom_point() +
  geom_segment(aes(x = -ei[1]*P[1, 1], y = -ei[1]*P[2, 1], xend = ei[1]*P[1, 1], yend = ei[1]*P[2, 1]),
               arrow = arrow(ends = 'both', length = unit(0.3, "cm")),
               color = "red", linewidth = 1.2) +
  geom_segment(aes(x = -ei[2]*P[1, 2], y = -ei[2]*P[2, 2], xend = ei[2]*P[1, 2], yend = ei[2]*P[2, 2]),
               arrow = arrow(ends = 'both', length = unit(0.3, "cm")),
               color = "blue", linewidth = 1.2) +
  theme_classic()
```

Nota Bene: The covariance matrix is a symmetric real matrix; hence, not only is it diagonalizable, but its eigenvectors form an orthogonal basis. Thus, in the plane, the second eigenvector is necessarily perpendicular to the first. Its direction does not provide much information; only its magnitude does. See the next section for a more involved method.

We can now reveal what my shape originally was:

```
reveal_shape()
```

You can also play with a second shape using `in_my_shape2` and `reveal_shape2`.

Nota Bene: This game also works perfectly in higher dimensions, of course! For instance, in dimension 3, the first eigenvector of the covariance matrix will indicate the fundamental direction, then the second one will indicate which direction in the plane orthogonal to the first is the most important, and so on. The relative sizes of the eigenvalues indicate the relative importance of these directions.

3.2.3 Independent Component Analysis

Obviously, there exists a method that is far more powerful (and more costly) to describe the fundamental axes of a point cloud, namely *Independent Component Analysis*. We will not present the mathematical details of this method, but only give the  implementation. Please refer to Wikipedia or any good book to learn more about the subject.

To run the following, you will need a package, so execute once `install.packages("ica")`. The value `nc` is the number of components, which is basically the dimension. There is a lot of information in the result of an independent component analysis, but we only care about `M`, the matrix whose columns are the directions of the components, and `vafs`, which is the *variance-accounted-for* by each component, i.e. the relative importance of each direction. Note that the result is often more accurate.

```
#install.packages("ica")
library(ica)
?ica

fit = ica(MonteCarlo, nc = 2, method = "fast")
View(fit)

P = fit$M
vafs = fit$vafs

ggplot(MonteCarlo, aes(x = x, y = y)) +
  geom_point() +
  geom_segment(aes(x = -vafs[1]*P[1, 1], y = -vafs[1]*P[2, 1], xend = vafs[1]*P[1, 1], yend = vafs[1]*P[2, 1]),
               arrow = arrow(ends = 'both', length = unit(0.3, "cm")),
```

```
            color = "red", linewidth = 1.2) +
  geom_segment(aes(x = -vafs[2]*P[1, 2], y = -vafs[2]*P[2, 2], xend = vafs[2]*P[1, 2], yend = vafs[2]*P[2, 2]),
            arrow = arrow(ends = 'both', length = unit(0.3, "cm")),
            color = "blue", linewidth = 1.2) +
  theme_classic()
```

3.3 Verifying Wigner semi-circle law

Recall that an *eigenvalue* of a matrix A is a number λ such that there exists a non-zero vector x satisfying $Ax = \lambda x$. There are at most n eigenvalues, which may be complex numbers even if A has only real coefficients, but if A is a real symmetric matrix, then A has exactly n real eigenvalues (counted with multiplicities).
The aim is to verify experimentally the following statement from Eugene Wigner:

> Let W_n be a random symmetric $n \times n$ matrix, e.g. a symmetric matrix whose entries are normally distributed, and let λ_n be an eigenvalue of W_n chosen at random (uniformly) among all eigenvalues of W_n. Then, asymptotically, up to normalization, λ_n follows a *Wigner semi-circle distribution* whose density function is the curve of a semi-circle, i.e. $t \mapsto \frac{2}{\pi}\sqrt{1-t^2}$ for $t \in [-1, +1]$, and 0 elsewhere (without normalization, one can use the general semi-circle distribution with density $t \mapsto \frac{2}{R^2\pi}\sqrt{R^2-t^2}$ for $t \in [-R, +R]$, and 0 elsewhere, for some radius R).

In particular, looking at all the eigenvalues of W_n at once for large n should give n samples from the Wigner semi-circle distribution (these samples are not independent, for you will see it works fine). Hence, drawing the histogram of the eigenvalues of W_n should yield a semi-circle. We want to verify this fact and try to check that we indeed have convergence.

3.3.1 Matrices and histograms

We have seen matrices (a bit). Of course, we can create matrices made from samples and visualize them!

```
n = 50
A = matrix(rnorm(n^2), n, n)

View(A)
plot(A)
image(A)
image(A, col = heat.colors(100))
image(A, col = topo.colors(100))
image(A, col = colorRampPalette(c("white", "skyblue", "navy"))(100))
```

Getting the spectral decomposition of a matrix is easy; reading it is slightly harder, because it contains both the eigenvalues and the eigenvectors, possibly as complex numbers.

```
Ei = eigen(A)
View(Ei)
```

Using `Ei$values` yields the vector of eigenvalues. Note that running `eigen(A)` several times yields the same result, but running the line `A = matrix(rnorm(n^2), n, n)` changes all the subsequent results.

First, we need to create a random **symmetric** matrix. There are plenty of ways to do it; the simplest one is to compute $A + A^T$ where A is a random matrix (A^T is the transpose of A).

Q (math): If A is a random matrix whose entries are independent and normally distributed variables with a common mean and variance, justify that $A+A^T$ is a symmetric matrix whose entries are normally distributed with a common mean and variance.

Q (math): Show that this is not the case when replacing $A+A^T$ by $A \cdot A^T$, even though $A \cdot A^T$ is symmetric.

Now, we sample such a random symmetric matrix and visualize its eigenvalues. The argument `binwidth` indicates that the width of each column shall be 3: R will take care of starting and ending the bins at the correct spot.

```
n = 500
A = matrix(rnorm(n^2), n, n)
W = A + t(A)

Ei = eigen(W)$values

ggplot(data.frame(ei = Ei), aes(x = ei)) +
  geom_histogram(binwidth = 3, fill = "lightblue", color='black')

# alternatively without ggplot:
# hist(Ei, probability = TRUE)
# ?hist
```

3.3.2 Finding the growth of the radius

Q: Increase and decrease `n = 500` in the previous code, and observe that the limits of the x-axis change according to `n`.

This would be very annoying if we want to compare the distributions of the eigenvalues of W_n for different n (e.g. to understand the convergence phenomenon), and it will moreover start to generate very large values, which will both mess up our graphical visualization and impact our computational speed. Our first task is to find the growth rate of this x-range, that is to say, to determine $\max \mathrm{Spec}(W_n)$ as a function of n. To this end, we sample one matrix of size n for different n: we need to be clever here and not take all n from 1 to (say) 100 because it would be too costly and may not go high enough. Here, I chose to work with sizes $n = \lfloor 1.5^k \rfloor$ for $k \in \{4, 19\}$ because I want a geometric growth, but not too fast.

```
test_range = data.frame(n = as.integer(1.5^(4:19)))
for (i in 1:16) {
  n = test_range$n[i]
  A = matrix(rnorm(n^2), n, n)
  W = A + t(A)
  test_range$max_ei[i] = max(eigen(W)$values)
}

View(test_range)
```

The column `max_ei` is increasing with respect to n, as we feared, but how fast? To understand this, we can plot it, especially in a log-log scale.

Nota Bene: Note that we sampled one matrix per size n, so our experimental data might be very noisy. If we do not manage to see anything in the upcoming visualization, then we should sample several matrices for each size n and compute the mean of our observation for each n.

```
ggplot(test_range, aes(x = n, y = max_ei)) +
  geom_line() +
  theme_classic()

ggplot(test_range, aes(x = n, y = max_ei)) +
  geom_line() +
  scale_x_log10() +
  scale_y_log10() +
  theme_classic()
```

This seems to be a line of slope $\frac{1}{2}$. This means that, up to some constant c, we should have $\log(\max \mathrm{Spec}(W_n)) = \frac{1}{2}\log n + c$, or equivalently that $\max \mathrm{Spec}(W_n)$ is proportional to $\sqrt{n}$. Hence, dividing `W` by $\sqrt{n}$ should keep the eigenvalues in a bounded range. By convention, we take the normalization $\frac{1}{2\sqrt{2n}}W_n$, so that the eigenvalues lie (almost surely) between -1 and $+1$. Remember that this is a mathematical experiment, not a proof of anything: if you want a proof, use pen and paper instead!

Let us check our normalization.

```
test_range = data.frame(n = as.integer(1.5^(4:19)))
for (i in 1:16) {
  n = test_range$n[i]
  A = matrix(rnorm(n^2), n, n)
  W = (A + t(A)) / (2 * sqrt(2 * n))
  test_range$max_ei[i] = max(eigen(W)$values)
}

ggplot(test_range, aes(x = n, y = max_ei)) +
  geom_line() +
  theme_classic()

ggplot(test_range, aes(x = n, y = max_ei)) +
  geom_line() +
  scale_x_log10() +
  scale_y_log10() +
  theme_classic()
```

3.3.3 Implementing the Wigner semi-circle law

We have our histogram, properly scaled: we need to construct the distribution that we want to compare it with. To this end, we need to define a *function*. If you have seen functions in other languages (Python, Java, C, etc.), then you should not be surprised. Otherwise, this is not the place to detail the fundamental construction, so we will say very little about it.

A function takes some arguments and returns a value. In between, you can perform as many operations as you want; the content of these operations will not be accessible outside the function. A function (as well as a number, a graphic, a table, etc.) can be stored in a named variable. To declare the arguments `x`, `y`, and `z` of a function, we just write `function(x, y, z)`. If we want to indicate a default value for an argument, say we want the value of `z` to be 3 if nothing else is chosen by the user, we just write `z = 3` inside the function declaration. By default, R returns the last computed value inside a function, so you can write `return(y)` at the end or just `y`; it will have exactly the same effect.

That being said, we want to construct the density function and the distribution of the Wigner semi-circle distribution. As usual, we name them `dwigner` and `pwigner`. The mathematical formula for `dwigner` has been explained at the beginning of this section, and the formula for `pwigner` is obtained via a straightforward integration.

```
dwigner <- function(x, radius = 1){
  y = numeric(length(x))
  inside = abs(x) <= radius
  y[inside] = (2 / pi*radius^2) * sqrt(radius^2 - x[inside]^2)
  return(y)
}

pwigner <- function(x, radius = 1){
  y = numeric(length(x))
```

```
  inside = abs(x) <= radius
  y[x <= radius] = 0
  y[x >= radius] = 1
  z = x[inside] / radius
  y[inside] = 1/2 + (z*sqrt(1 - z^2) + asin(z))/pi
  y # return(...) is useless in R because it automatically returns the last computed value, here 'y'
}
```

Q: To check that everything works, we draw the density and the distribution. What properties are you expecting to see in these graphics?

```
ggplot(data.frame(x = c(-1.2,1.2)), aes(x = x)) +
  stat_function(fun = dwigner, linewidth = 0.5, color='blue') +
  stat_function(fun = pwigner, linewidth = 0.1, color='red') +
  labs(title = "Density-and-distribution-of-Wigner(0,1)", y = "Density,-Distribution") +
  ylim(0, 1.1) +
  theme_classic()
```

3.3.4 Checking that we have the correct limit distribution

We draw the density of the Wigner semi-circle distribution together with our histogram of eigenvalues (using an aesthetic that specifies that the histogram should display densities rather than counts). Observe how well they coincide!

```
n = 500
A = matrix(rnorm(n^2), n, n)
W = (A + t(A)) / (2 * sqrt(2*n))
Ei = eigen(W)$values

ggplot(data.frame(ei = Ei), aes(x = ei)) +
  geom_histogram(aes(y = after_stat(density)), fill = "lightblue", color='black') +
  stat_function(fun = dwigner, linewidth = 0.5, color='blue') +
  labs(title = "Density-of-Wigner(0,1)-versus-histogram-of-sample", y = "Density,-Distribution") +
  xlim(-1.1, 1.1) + ylim(0, 1) +
  theme_classic()
```

3.3.5 Estimating the speed of convergence (in distribution)

First and foremost, we can check whether the expectation and variance (of the eigenvalues of normalized symmetric random matrices) converge when $n \to +\infty$. The expectation of the Wigner semi-circle distribution is 0 (you can clearly see the symmetry in the graph), and the variance is $\frac{1}{4}$ (for radius 1), which can be computed directly via an integral. Let's check this on one example.

```
mean(Ei) # 0 ?
var(Ei) # 1/4 ?
```

Going further, we construct a table `test_convergence`, with the same sample of sizes as before (see the numerous comments in **??**, notably we sample only one matrix per size, which is not ideal). Then we plot it and observe the speed of convergence.

```
test_convergence = data.frame(n = as.integer(1.5^(4:19)))
for (i in 1:16) {
  n = test_convergence$n[i]
  A = matrix(rnorm(n^2), n, n)
  W = ( A + t(A) ) / (2 * sqrt(2*n))
```

```
  Ei = eigen(W)$values
  test_convergence$mean[i] = mean(Ei)
  test_convergence$var[i] = var(Ei)
}
View(test_convergence)

ggplot(test_convergence, aes(x = n)) +
  geom_line(aes(y = mean, color = 'mean')) +
  geom_line(aes(y = var, color = 'variance')) +
  labs(title = "Convergence of mean and variance of eigenvalues", y = 'means and variances') +
  theme_classic()
```

Further, we can compute for each n the *Kolmogorov distance*, that is $\sup_t |F_n(t) - F(t)|$. The software R provides many ways of comparing samples and distributions, including the Kolmogorov distance. The standard approach is to compute the Kolmogorov–Smirnov test, named `ks.test`.

```
?ks.test
KS = ks.test(Ei, pwigner)
View(KS)
```

Some clarification of the results of `ks.test(sample, F)`, where the sample is interpreted empirically and `F` as a distribution function:

- `D` $= \sup_t |G(t) - F(t)|$, where $G(t)$ is the empirical cumulative distribution function of the sample
- `p-value` $= 1 -$ (answer to "If `Ei` were truly distributed according to `dwigner`, how surprising is the observed data?")
- `two-sided` means that R tests whether the empirical distribution deviates from `pwigner` in either direction
- `ks.test` uses an asymptotic approximation; it is not computing an exact `D`, but a very accurate estimate

We only need `D` here, so let's extract it and add a new column to our table `test_convergence`.

```
D = KS$statistic
D

test_convergence = data.frame(n = as.integer(1.5^(4:19)))
for (i in 1:16) {
  n = test_convergence$n[i]
  A = matrix(rnorm(n^2), n, n)
  W = ( A + t(A) ) / (2 * sqrt(2*n))
  Ei = eigen(W)$values
  test_convergence$D[i] = ks.test(Ei, pwigner)$statistic
}
View(test_convergence)

ggplot(test_convergence, aes(x = n, y = D)) +
  geom_line() +
  labs(title = "Convergence of the Kolmogorov distance") +
  theme_classic()

ggplot(test_convergence, aes(x = n, y = D)) +
  geom_line() +
  scale_x_log10() + scale_y_log10() +
  labs(title = "Convergence of the Kolmogorov distance in log−log") +
  theme_classic()
```

We can be satisfied: this decreases quite rapidly to 0! More precisely, it seems that D_n is roughly proportional to 1.75^{-n} (on my computer).

3.4 Testing a convergence in probability? a convergence almost sure?

That is an excellent project: you should definitely do it... on your own!

4 Course 4: Fitting

4.1 Lagrange interpolation

4.1.1 Lagrange interpolating polynomials in theory

Given points $(a_1, b_1), \dots, (a_n, b_n) \in \mathbb{R}^2$, what is the polynomial P of smallest degree that goes through these points, i.e. such that $P(a_k) = b_k$ for all $k \in [n]$?

This question is directly answered by the theory of *Lagrange interpolating polynomials*: there is a unique polynomial P of degree $n-1$ that goes through these points. To construct such a polynomial P, first define $L_i = \frac{1}{\prod_{j \neq i}(a_i - a_j)} \prod_{j \neq i}(X - a_j) \in \mathbb{R}_{n-1}[X]$ for each $i \in [n]$. Then, one can easily verify that a solution is:

$$P = \sum_{i=1}^{n} b_i \cdot L_i$$

This solution is unique (inside $\mathbb{R}_{n-1}[X]$) because $(L_1, \dots, L_n)$ forms a basis of $\mathbb{R}_{n-1}[X]$ (as long as the $a_1, \dots, a_n$ are different).

4.1.2 Lagrange interpolation in R

With R , you can directly give points and ask for the interpolating polynomial. R will define the associated Lagrange interpolating polynomials, and deduce the interpolating polynomial from them.

To this end, we use the package `PolynomF`.

```
install.packages("PolynomF")

library(PolynomF)
```

We can either ask for the polynomial that takes the value 0 (i.e. has roots) for $i \in \{0, \dots, 5\}$; or interpolate the exponential function so that our interpolation is exact for $x \in \{0, \dots, 5\}$.

```
?poly_calc

p = poly_calc(0:5)
plot(p, col = 'blue')

p = poly_calc(0:5, exp(0:5))
plot(p, col = 'blue')
curve(exp, add = TRUE, col = "red")
```

4.1.3 Checking Faulhaber's formula and getting the Bernoulli numbers

One reasonable idea when performing polynomial interpolation is to split data into two sets of points: one that you use for constructing the interpolator (the *training set*), and one that you use to check that your interpolator is somewhat correct (the *validation set*, or *test set*). If you expect the relation between x and y to be a polynomial of degree d, then your training set should contain at least (or preferably "exactly") $d+1$ points.

We want to verify the following well-known formula:

$$\sum_{k=1}^{n} k^2 = \frac{n(n+1)(2n+1)}{6}$$

More generally, for all $q \in \mathbb{N}$ there exists a polynomial F_{q+1} of degree $q+1$, the *Faulhaber* polynomial, such that:

$$\sum_{k=1}^{n} k^q = F_{q+1}(n)$$

Q: Create a `data.frame` named `fr` whose first column contains $n \in \{1, \dots, 100\}$, and the second column contains n^2.

Q: Using the function `cumsum` (for *cumulative summation*), add a new column to your `data.frame`.

Q: Compute the interpolating polynomial P for $\left(n, \sum_{k=1}^{n} k^2\right)$ for $n \in \{1, \dots, 3+1\}$. Read the polynomial and be happy.

Q: Add a column `check` to your `data.frame` whose values are obtained by applying P to the column `n`. Compare the column `cumul` and the column `check`, and be happy.

Q: To be sure, run the following test:

```
all(fr$cumul == fr$check)
```

Q: Re-write your script to handle $\sum_{k=1}^{n} k^q$ for any q. Run it for $q = 5$, and understand that there is a problem...

This problem comes from numerical errors. You would like to have integer numbers, but R finds your numbers too difficult, so it prefers storing them as `doubles` (or `dbl` for short), which are real numbers with a certain precision. Consequently, when checking if two numbers are equal (that is, if their difference is 0), it might be that their difference is 10^{-16}, but not 0. To solve this conundrum, let us improve our test by allowing some *tolerance* in the exactness of the computations:

```
all.equal(fr$cumul, p(fr$n), tolerance = 1e-2)
```

Q: Run your code for $q = 13$, read the first coefficient of the interpolating polynomial, and complain about a problem. Sadly, solving this problem goes beyond the current method (i.e. R is not made for this kind of computations).

We will work with $q \leq 10$ in what follows.

Q: Create a list of all the Faulhaber polynomials $F_1, \dots, F_{11}$. Draw these polynomials on the interval $[0, 1]$.

Your final code should look like:

```
N = 12
qvals <- seq(0, 1, length.out = N+1)
cols <- colorRampPalette(c("blue", "red"))(length(qvals))

Faulhaber = list()

plot.new()
plot(0:1, 0:1, type="n")
for (q in 0:N) {
  print(q)
  Nq = n^q
  fr = data.frame(n, Nq)
  fr$cumul = cumsum(fr$Nq)
  p = poly.calc(1:(q+2), fr$cumul[1:(q+2)])
  Faulhaber = append(Faulhaber, p)
  print(p)
  curve(p, from=0, to=1, xlab=q, n = 501, col = cols[q], add=TRUE)
  print(all.equal(fr$cumul, p(fr$n), tolerance = 1e-2))
  cat("\n") # print an empty line
}
```

Q: The coefficients of F_{q+1} in front of n are the Bernoulli numbers (of second kind), denoted B_q. Draw the sequence $B_1, \dots, B_{10}$.

Q: Does the sequence of Bernoulli numbers diverge to $+\infty$ (as it should be), according to your experiment?

Your final code should look like:

```
Bernoulli = data.frame(q = c(0:N))
for (q in 0:N) {
  Bernoulli$num[q+1] = coefficients(Faulhaber[q+1])[1+1]
}

plot(x = Bernoulli$q, y=Bernoulli$num)
```

This exercise should have convinced you that (exact) polynomial interpolation in R is a not-so-good idea.

4.2 Linear fits

A *linear fit* to explain the observations $\boldsymbol{y} = y_1, \dots, y_m$ from the variables $\boldsymbol{x} = x_1, \dots, x_n$ is a matrix $A \in \mathrm{Mat}_{m,n}(\mathbb{R})$ and a vector $\boldsymbol{b} \in \mathbb{R}^m$. Given points $(\boldsymbol{x}_1, \boldsymbol{y}_1), \dots, (\boldsymbol{x}_r, \boldsymbol{y}_r)$, the "best" linear fit is the one minimizing the sum-of-squares:

$$\sum_{k=1}^{r} \big(\boldsymbol{y}_k - (A\boldsymbol{x}_k + \boldsymbol{b})\big)^2$$

R has a direct built-in function for linear fits. As a (stupid) exercise, we want to find, experimentally, the following formula:

$$\mathbb{E}U = \frac{m+M}{2} \qquad \text{for } U \sim \mathcal{U}nif([m, M])$$

4.2.1 Univariate linear fits

As the most basic (and most useful) example, we consider the case $m = n = 1$, and denote a instead of A for the regression matrix (since it is actually just a number).

Q: Create a vectorized function `UniMean` with arguments `m`, and `M` that computes the mean of 100 samples of the uniform distribution on $[m, M]$.

```
UniMean = Vectorize(function(a, b) mean(runif(100, min = a, max = b)))
```

Q: Create a `data.frame`, named `df` with columns: `M = c(1:100)` and `Y = UniMean(-10, M)`.
Q: Compute the following, and try to understand both `View(model)` and `summary(model)`.

```
M = c(1:100)
df = data.frame(M)
df$mean = UniMean(a = −10, b = M)

model = lm(mean ~ M, data = df)
View(model)

ggplot(df, aes(x = M, y = mean)) +
  geom_point(shape=1, show.legend=FALSE) +
  geom_smooth(method="lm", se=TRUE, show.legend=FALSE)
```

Q: In particular, the *coefficient of determination* R^2 can be accessed using `summary(model)$r.squared` (respectively, `summary(model)$adj.r.squared` for the adjusted coefficient of determination). Intellectually, one should think of R^2 as the correlation between the variable and the observation, i.e. $R^2 = \frac{\mathrm{cov}(X,Y)}{\mathbb{V}X \cdot \mathbb{V}Y}$. In practice, R^2 is computed as:

$$R^2 = \frac{\sum_{k=1}^{r} \big(y_k - f(x_k)\big)^2}{\sum_{k=1}^{r} (y_k - \overline{y})^2} \qquad \text{where } f : x \mapsto a \cdot x + b$$

where $\overline{y}$ is the average of $y_1, \ldots, y_r$. The closer R^2 is to 1, the better the linear fit axplains the data. A value such as $R^2 = 0.7$ may be interpreted as follows: "70% of the variance in the response variable can be explained by the explanatory variables. The remaining thirty percent can be attributed to unknown, lurking variables or inherent variability." If R^2 is less than 0.7, you should typically pause and ponder: do you really believe that your data befits by a **linear** model?

4.2.2 Multivariate linear fits

Let's now prove experimentally the formula: $\mathbb{E}U = \frac{m+M}{2}$ for $U \sim \mathcal{U}nif([m, M])$.

Q: Create another vector `m = c(-1:-100)`. Using `expand.grid(...)`, construct a `data.frame` with two columns `m` and `M` such that the rows contain all possible values (x_1, x_2) for $x_1 \in m$ and $x_2 \in M$.

```
m = c(-1:-100)
M = c(1:100)
df = expand.grid(m = m, M = M)
View(df)
```

Q: Add a column `mean` to your `data frame` which is `UniMean(x1, x2)` for all possible $(x_1, x_2) \in m \times M$.

```
df$mean = UniMean(a=df$m, b = df$M)
```

Q: Run the following, and understand the result using `View(model)` and `summary(model)`:

```
model = lm(Y ~ m + M, data = df)
```

Q: Are you happy with the result? What is the coefficient of determination and what does it tell you?

Q: Use a 3D plot to visualize image of your data.

```
# install.packages("plot3D")
library(plot3D)

scatter3D(x=df$m, y=df$M, z=df$mean, bty = "b2")
```

4.3 A first glimpse at training sets versus validating sets

This section and the following sections were inspired by the excellent course:
https://harvard-iacs.github.io/2018-CS109A/lectures/lecture-6/presentation/Lecture6_MR_ModelSelection.pdf

Q: Redo the previous table but for the variance instead of the average of $\mathcal{U}nif([-10, M])$. What do you think of the result?

```
UniVar = Vectorize(function(a, b) var(runif(100, min = a, max = b)))
df$Var = UniVar(a=df$m, b = df$M)

model3 = lm(Var ~ m + M, data=df)
View(model3)
summary(model3)$r.squared
summary(model3)$adj.r.squared

scatter3D(x=df$m, y=df$M, z=df$Var, bty = "b2")
```

Q: Make a linear fit for the variance using only `m = c(-1:-5)` and `M = c(1:5)`. Look at the coefficients obtained from this regression, and construct a linear map using them, as follows.

```
training_set = df[df$m >= -5 & df$M <= 5, ]
# training_set = sample(nrow(df), nrow(df)/20)
dim(training_set)

model4 = lm(Var ~ m + M, data=training_set)
View(model4)

summary(model2)$r.squared
summary(model2)$adj.r.squared
```

The coefficient of determination is already computed on the data you have given, but its definition is not restricted to be used on the data points of the training set: you can use it on any set of points.

Q: In particular, compute R^2 where $(x_1, y_1), \ldots, (x_r, y_r)$ designate **all** the points of your data set (both training and validating, i.e. `m = c(-1:-100)` and `M = c(1:100)`), but where f is the function defined by your model fitted on your training set only. Comment the result.

```
b = coef(model4)
b

f <- Vectorize(function(x1, x2) {
  unname(b[1] + b["m"] * x1 + b["M"] * x2)
})
sse = sum((f(df$m, df$M) - df$Var)^2)
ssr = sum((f(df$m, df$M) - mean(df$Var))^2)
ssr / (sse + ssr)
```

Q: Perform the same exercise but using random rows of `df` instead (say use 5% of the data). Comment on the new R^2.

4.4 Polynomial fits

Given data points $(x_1, y_1), \ldots, (x_r, y_r)$, the problem of determining a polynomial $P(t) = \sum_{i=0}^{d} \beta_i t^i$ of a given degree d which best fits these data points (i.e $y_i \approx P(x_i)$ for all i) is still a linear problem: we are looking for the coefficients of P which are not raised to any power! We can formulate the problem as follows:

For given d and $(x_1, y_1), \ldots, (x_r, y_r)$ with $r \geq d+1$, determine the numbers $\boldsymbol{\beta} = \beta_0, \ldots, \beta_d$ that minimize $\|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\beta}\|^2$ where:

$$\boldsymbol{X} = \begin{pmatrix} 1 & x_1^1 & x_1^2 & \ldots & x_1^d \\ 1 & x_2^1 & x_2^2 & \ldots & x_2^d \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_r^1 & x_r^2 & \ldots & x_r^d \end{pmatrix}$$

The solution is (using Euclidean norm) (invertibility is ensured by Vandermonde determinants):

$$\boldsymbol{\beta} = \left(\boldsymbol{X}^\top \boldsymbol{X}\right)^{-1} \boldsymbol{X}^\top \boldsymbol{y}$$

Of course, polynomial fits are directly implemented in R. Actually, more generally, any fit that boils down to a linear regression can be obtained using `lm(Y ~ ..., data = df)`, by completing the command accordingly. To get a polynomial in the variable `X`, for example, we proceed as follows:

```
model = lm(Y ~ poly(X, d))
```

But first, we need some data to train on and validate against. Let's make our own (you can download my script directly)! We fix a polynomial `p` (here of degree `D = 5`) by sampling some coefficients (here, we fix the coefficients $\alpha_0, \ldots, \alpha_D$ so that everyone works with the same polynomial).

Then, we take some random `X`, and compute `Y = p(X)` but we add random noise to each data point (normally distributed, here $\mathcal{N}(0, 0.75^2)$).

```
make_poly <- function(coef) {
  Vectorize(function(x) {
    sum(coef * x^(seq_along(coef) - 1))
  })
}

noisified_data <- function(f, x) {
  f(x) + 0.75*rnorm(length(x))
}

D = 5
# alpha = rnorm(D+1)
alpha = c(0.152866375, 1.257512796, -0.144738591, 0.339900781, 0.143917067, 0.002062239)

p = make_poly(alpha)
p(5)
p(0:4)

p(0) == alpha[1] # Sanity check
p(1) == sum(alpha) # Sanity check
```

Finally, we can prepare our data set as follows:

```
nbr = 10
X = rnorm(nbr)
Y = noisified_data(p, X)

plot(x = X, y = Y)
```

Q: Try fitting the data using a linear fit, then using a polynomial fit of degree 5, and then of degree 9. What were you expecting? Comment the result.

Q: Print the coefficients of `model = lm(Y ~ poly(X, 5, raw=TRUE))`. *NB*: R uses (cleverly) an orthonormal basis of polynomials, instead of the standard basis $1, t, t^2, \ldots$. This speeds up the computations but makes it harder to access the coefficients of the polynomial. Using `raw = TRUE` as an argument forces R to use the standard basis instead of the orthonormal one.

```
model = lm(Y ~ X)
summary(model)$r.squared

model = lm(Y ~ poly(X, 5))
summary(model)$r.squared

model = lm(Y ~ poly(X, 9))
summary(model)$r.squared

model = lm(Y ~ poly(X, 5, raw=TRUE)) # Augment nbr to get more accuracy
coef(model)
```

More generally, we can fit polynomials of degree from 1 to 9.

```
d_max = nbr - 1

qvals = seq(0, 1, length.out = d_max+1)
cols = colorRampPalette(c("blue", "red"))(length(qvals))
```

```
poly_estimate = data.frame(d = 1:d_max)
for (d in 1:d_max) {
  print(d)
  model = lm(Y ~ poly(X, d))

  poly_estimate$Rsquared[d] = summary(model)$r.squared

  estim_p = Vectorize(function(x){predict(model, newdata = data.frame(X = x))})
  curve(estim_p, from=min(X), to=max(X), xlab=d, n = 501, col = cols[d], add=TRUE)
}

View(poly_estimate)
plot(x = poly_estimate)
```

There are a lot of stuffs to comment in what you have just witnessed!

1. We clearly do not have enough data points: we know that the actual polynomial has degree 5, but R^2 for `d = 5` is not very high (the exact value depends on the sampled points `X` you have).
2. A higher-degree polynomial usually provides a better R^2. When `d = nbr -1`, we actually get a polynomial that perfectly fits our data points. Indeed, this is simply Lagrange polynomial interpolation.
3. However, this leads to overfitting problems:
   - Computation time becomes high,
   - Coefficients values become extreme (use `View(lm(Y ~ poly(X, 9, raw = TRUE)))` to convince yourself).
   - Numerical errors tend to snowball (see our discussion on Faulhaber's polynomials in Section 4.1.3)
   - Clearly, our graph does not "fit" our data, in the sense that the resulting polynomial performs very poorly at predicting the values of points outside our training set (see the next sections).

4.5 Basics of model selection

If we want to select the "best" model to explain our data, we would need to split our data set into a training set and a validation set, then establish the model on the training set and check its performance on the validation set. If we have r data points and N different models to test, there are $O(N \cdot 2^r)$ ways of splitting the data set and modeling it: this would be far too costly.

There are many many many ways to solve this cost conundrum, and it heavily depends on your type of data, on exterior assumptions that you may have, etc. We will see one way to do it that befits our problem (i.e. estimating the polynomial seen in the previous section).

4.5.1 Even/odd splitting of data, and coefficient of determination

Q: Select 100 points uniformly at random in $[-2.5, +2.5]$ and sort them. Construct the noisy evaluation of the (polynomial) function at stake. Make a `training_set` out of the points (x_{2k}, y_{2k}) and a `validating_set` out of the points (x_{2k-1}, y_{2k-1}).

This kind of splitting is particularly suitable for our type of problem and our data set: we fit the model on some points and then verify that it behaves correctly in-between these points. Moreover, we know that we have a polynomial, so is it very important to have points that are "far away from $x = 0$" so that the leading monomial dominates the formula and can be captured by the model. We chose our training and validation sets to have the same size, but this is completely arbitrary.

```
nbr = 100
X = runif(nbr, -2.5, 2.5)
```

```
X = sort(X)
Y = noisified_data(p, X)
plot(x = X, y = Y)

training_set = data.frame(X = X[2 * 1:(nbr/2)], Y = Y[2 * 1:(nbr/2)])
validating_set = data.frame(X = X[-1 + 2 * 1:(nbr/2)], Y = Y[-1 + 2 * 1:(nbr/2)])

dim(training_set)
dim(validating_set)
```

Q: For polynomial fits of degree between 1 and 15, compute the R^2 of the training set and of the validation set (this might crash for high enough `d` due to the training points been too close together: do not bother). Store the result in a table `poly_estimate`. Moreover, for `d = 5`, plot the curves of the actual polynomial and of the estimated polynomial on top of the data points.

```
d_max = 15 #nbr/2 - 1
poly_estimate = data.frame(d = 1:d_max)
for (d in 1:d_max) {
  print(d)
  model = lm(Y ~ poly(X, d), data = training_set)

  if (d == D) {
    plot(x = X, y = Y)
    estim_p = Vectorize(function(x){predict(model, newdata = data.frame(X = x))})
    curve(estim_p, from=min(X), to=max(X), xlab=d, n = 501, col = 'red', add=TRUE)
    curve(p, from=min(X), to=max(X), xlab=d, n = 501, col = 'blue', add=TRUE)
    }

  poly_estimate$Rsquared_training[d] = summary(model)$r.squared

  estim_p = Vectorize(function(x){predict(model, newdata = data.frame(X = x))})
  sse = sum((estim_p(validating_set$X) - validating_set$Y)^2)
  ssr = sum((estim_p(validating_set$X) - mean(validating_set$Y))^2)
  poly_estimate$Rsquared_validating[d] = ssr / (sse + ssr)
}

View(poly_estimate)
```

Q: Plot the resulting `poly_estimate` to visualize both `Rsquared_training` and `Rsquared_validation`. Comment the result.

```
plot(x = poly_estimate$d, y = poly_estimate$Rsquared_training, type="l", col="blue")
points(x = poly_estimate$d, y = poly_estimate$Rsquared_training, col="blue")
points(x = poly_estimate$d, y = poly_estimate$Rsquared_validating, col="red")
lines(x = poly_estimate$d, y = poly_estimate$Rsquared_validating, col="red")
```

There exists of course many other metrics that can be used for assess the predictive quality of models on both the training set and the validation set, but we will not enter the details here.

4.5.2 Araike and Bayesion information criteria

Rather than validating the model on a separate validation set, we can compare different models by attributing them a score depending on how well they balance using few parameters (*model complexity* criterion) and estimating the data correctly (*goodness-of-fit* criterion). We see two such scores here.

The *Akaike information criterion (AIC)* of a model is given by:

$$\mathrm{AIC} = 2k - 2\ln L$$

where k is the number of parameters of the model (for a polynomial of degree d, we have $k = d+1$ or $k = d+2$ depending on whether the model also estimate the variance of the data set), and where L is the maximum likelihood (for our model, and for usual applications, this maximum likelihood is computed under the assumption that the actual data represent some underlying "true" values plus an error which is normally distributed, so the computation reduces to a sum of squares in our case).

The *Bayesian information criterion (BIC)*, defined by Gideon Schwarz, of a model is given by:

$$\mathrm{BIC} = k \ln r - 2 \ln L$$

where r is the number of data points used to train the model.

Both AIC and BIC give scores which have no meaning by themselves, but become useful for comparing several models: the lower the AIC or BIC, the better the model.

Q: Add columns `aic` and `bic` to your table `poly_estimate` and plot them. Comment the results.

```
d_max = 15 #nbr/2 - 1
for (d in 1:d_max) {
  print(d)
  model = lm(Y ~ poly(X, d), data = training_set)

  poly_estimate$aic[d] = AIC(model)
  poly_estimate$bic[d] = BIC(model)
}
View(poly_estimate)

plot(x = poly_estimate$d, y = poly_estimate$aic, type="l", col="purple")
points(x = poly_estimate$d, y = poly_estimate$aic, col="purple")
points(x = poly_estimate$d, y = poly_estimate$bic, col="green")
lines(x = poly_estimate$d, y = poly_estimate$bic, col="green")
```

4.5.3 Remark: fitting with other functions

Imagine you want to fit your data with a polynomial which has a zero coefficient on x (but non-zero on $x^2, \ldots, x^5$), or with an even polynomial, or with a trigonometric function, or with anything. In R, this is easy to do. For instance, try:

```
model = lm(Y ~ X^2 + X^3 + X^4 + X^5) # This is a good fit for our polynomial

model = lm(Y ~ cos(X) + sin(X))

model = lm(Y^2 / X ~ cos(X))
```

I believe you get the idea.

One small remark, though. Suppose that you have a named quantity, e.g. `d = 5`, and want to use it in a formula: if you write `Y ~ X^d`, it will not work directly because R does not understand whether `d` is a parameter of the model or is a constant defined externally. You need to tell R to first evaluate this formula before using it, using `I(...)`:

```
model = lm(Y ~ X^2 + I(X^d))
```

4.6 Using log-log scales

4.6.1 Fitting a power law

Sometimes, our data may not be well suited for a regression in their current form, but may become far more tamed if we first apply a well-suited function to either the variables or the observations. A

good example of such a transformation is the determination of a and b in:

$$y = a \cdot x^b$$

If your data satisfy a power law of this form, you could use a linear model to perform a polynomial fit, but this is costly, subject to numerical errors, and somewhat imprecise. However, if you take the logarithm on both sides, you get:

$$\log y = b \cdot \log x + \alpha \qquad \text{where } \alpha = \log a$$

This means that you perform a linear fit on $(\log x_1, \log y_1), \ldots, (\log x_r, \log y_r)$: the linear coefficient would be a good estimate of b, while the intercept coefficient would estimate a after taking the exponential. This method works even if b is not an integer, and it can be performed with your own eyes (rather than using a computer) by actually plotting the points $(\log x_1, \log y_1), \ldots, (\log x_r, \log y_r)$ and estimating the slope.

Q: Go back to Section 3.3.2 or to Section 3.3.5, and use a linear fit to obtain a more precise estimate of the power laws involved there.

```
plot(x = log(X), y = log(Y))

model = lm(log(Y) ~ log(X))
```

4.6.2 Recovering the degree of a polynomial? Not really...

Log-log scales (or semi-log scales) are extremely powerful for identifying power laws, with several variables, several observations, etc. One can imagine other scales, by taking logarithms several times.

However, there is a fundamental limit to this method. Let's consider the following problem: suppose that $y = P(x)$ for a certain polynomial $P(t) = \sum_{i=0}^{d} a_i t^i$ of degree d. We have seen in Section 4.5 how to compare several polynomial fits and thus how to determine the degree, but we would like to obtain it more directly.

Consider $\log(y) = \log(P(x))$. When $x \to +\infty$, we have $P(x) \sim a_d x^d$, so $\log(y) \sim d \cdot \log x + \log a_d$, so we can perform a linear fit and recover the coefficient d. However, in practice, how "large" shall x be to get a good estimate for d? Besides, as we are using log, all the values should be positive (or should be redressed to become positive).

Q: With our good old polynomial of degree 5, sample `X` between 5 and 10, and estimate the degree. Comment the result.

```
nbr = 100
X = runif(nbr, 5, 10)
Y = noisified_data(p, X)
plot(x = log(X), y = log(Y))

model = lm(log(Y) ~ log(X))
summary(model)$r.squared
coef(model)
```

That's... underwhelming.

Q: Redo the experiment for `X` between 50 and 75. Comment the result.

```
X = runif(nbr, 50, 75)
Y = noisified_data(p, X)
plot(x = log(X), y = log(Y))

model = lm(log(Y) ~ log(X))
summary(model)$r.squared
coef(model)
```

That's... underwhelming again.

The general theory holds, but the coefficient on t^5 in our polynomial is too small to be identified easily. To get the linear coefficient of our linear fit to be the true degree of our polynomial, we would need to go to very large values of x, but this would generate values of $P(x)$ that are too large to work with or would create numerical errors.

However, there is still one piece of good news: our estimation of d yields a lower bound for the true d (and this lower bound gets tighter as x increases). Indeed, the slope of the linear fit is an estimate of $\frac{\partial \log y}{\partial \log x}$ (since we fit $\log y$ against $\log x$), and:

$$d - \frac{\partial \log y}{\partial \log x} = d - \frac{x \cdot P'(x)}{P(x)} = \frac{\sum_{i=0}^{d}(d-i)a_i x^i}{P(x)}$$

In particular, if $P(x) > 0$ (which we need in order to compute $\log y = \log P(x)$) and if all $a_i \geq 0$ (which is sometimes the case for real-world data), then $\frac{\partial \log y}{\partial \log x} \leq d$ for all $x \geq 0$.

In general, if $P(x) > 0$ (even though $a_i \not> 0$, as long as $a_d > 0$), we have $\frac{\partial \log y}{\partial \log x} \leq d$ for sufficiently large x, but possibly very large.

4.7 Plenty of other fits

There are many more things to say about fitting data, it is a vast topic. If you are interesting in learning more, some relevant keywords may be (not restricted to):

- Nonlinear least squares (`nls`)
- Trigonometric fits
- Curve fitting
- Probability distribution fitting
- Bayesian linear regression

5 Course 5: χ^2 test, Student test, confidence intervals, etc.

Exam

Preamble

Welcome to the exam of R !

You are going to answer 2 of the following 5 exercises. You will produce a R script and send it by email at germain.poullot@uni-osnabrueck.de, before the $8^{\mathbf{th}}$ **of July 2026** (included). Nothing will be accepted after this date. I will run your script, print its results, and ask you questions during your oral exam (during between 5 and 10 minutes).

Your script will be named `John_Smith_exam_statistics.R` using your own name. Some questions require you commenting on something: you can either leave a comment in your script, using `#...`, or send a PDF together with your script (be careful, you are not allowed to send the script itself as a PDF, and the PDF should be named the same way as your script). "Comment the result" (and similar phrasing) means "expose your mathematical understanding of the result your code is yielding". Please write in English, and avoid manuscript text if you are unsure of your calligraphy. Short and efficient answers are preferable. If you write more than 5 lines of code for a question, you should pause and ponder.

You need to download the filed "Exam_data.R" and "Exam_data_3.csv" from the "Exam folder" of StuD.IP, and save it in the same folder as your future script. Open R , create a new script, and start your script by `source("Exam_data")`, then run `sanity_check()`. If you do not get `TRUE` as an answer, open manually the file "Exam_data.R", then copy-paste the whole file at the beginning of your script and run its lines one-by-one.

Start with Exercise 0: it will guide you on what to do.

Exercise 0

With R , sample two numbers uniformly between 1 and 6, then take their integer values. These two numbers designate the two exercises you have to answer.

In your script, copy-paste the result as a comment below the line which computes these two random numbers.

Exercise 1

The variable `primes` from the source file "Exam_data.R" contains all prime numbers less than 5 000 (except the number 2 which has been removed).

1. How many prime numbers are they less than 5 000?
2. Construct a `data.frame` named `P` with a column `n` ranging from 2 to the correct number, and a second column `p_n` with the content of the variable `primes`.
3. According to Hadamard–La-Vallée-Poussin theorem on prime numbers, the sequence $\frac{p_n}{\ln n}$ is asymptotically equivalent to n. Add a third column named `scaled_p_n` which is equal to $\frac{p_n}{\ln n}$.
4. Check Hadamard–La-Vallée-Poussin theorem by performing a linear fit on the following formula: `scaled_p_n` $\sim$ `n`. Give the coefficients of the fit, and the R^2.
5. It is claimed that there are as many prime numbers whose final digit is 1, as there are whose final digit is 3, as there are whose final digit is 7, as there are whose final digit is 9. Check this claim on the primes below 5 000 (you can use `%%` to take a modulus).

Oral question: How to run the linear model using only the first 10 primes? What do you expect on R^2?

Oral question: How to extract only the primes of the form $4k + 1$? How to check if 2 999 is prime?

Oral question: How to plot p_n as a function of n, in red?

Exercise 2

You have seen in the course that the expectation of the Cauchy distribution is infinite. R has built-in functions for the Cauchy distribution. You can sample from it using `rcauchy(...)`.

1. What are the names of the functions yielding the density function, the probability distribution, and the quantiles of the Cauchy distribution.
2. Construct a `data.frame` named `C` with column `N` ranging from taking values $k \in \{1, \dots, 1000\}$. Add two columns to `C`, named `Mean` and `Standard_Deviation` whose values are the means and the standard deviations of n samples of the Cauchy distribution (where n is the value in `C$N`). Be careful, for each row, take the mean and standard deviation of the *same* n samples. View the result.
3. Plot the means and the standard deviations you have obtained. Comment the result in regard to the first line of the exercise.
4. On the plot of the standard deviation, plot in red the cases where the mean is at least 10 in absolute value. Comment the result.
5. A friend tells you that if X is Cauchy distributed, then $1/X$ is also Cauchy distributed. Sample 1000 times the Cauchy distribution, and run a Kolmogorov–Smirnov test for the null hypothesis "$\frac{1}{x_1}, \dots, \frac{1}{x_{1000}}$ is Cauchy distributed.". Conclude.

Oral question: In the result of the `ks.test`, what does "two-sided" means? What does the `statistic` (i.e. `D`) indicates?

Oral question: How to plot the density of the Cauchy distribution between -1 and 1, in red? Draw what I should see.

Oral question: What is the domain of definition of `qcauchy`? If I execute `qcauchy(2)`, what would I get?

Exercise 3

The file "Exam_data_3.csv" is a .csv file containing 3-dimensional vectors, one vector per row. These vectors are samples of a normal distribution $\mathcal{N}(\boldsymbol{\mu}, \Sigma)$.

1. Read this .csv file and store its contain in a variable `Z`. How many samples are they? (If you cannot read this file in R, do Exercise 5 instead, and do Exercise 2 if you are already doing Exercise 5.)
2. Estimate $\boldsymbol{\mu}$ and Σ. What is a confidence region for $\boldsymbol{\mu}$ (no need to compute it: this is a math question)?
3. Looking at Σ, it seems that the two first coordinates have the same variance. Run and interpret a F-test, using `var.test(...)`, to verify this hypothesis.

4. Run a F-test between the first and the third coordinate and interpret the result.
5. Run `plot(Z)`: does what you see agree with the results of the previous questions?

Oral question: I have computed the eigenvalues of the estimated Σ, namely: $20.5, 2.47, 0.766$. Why are they positive?

Oral question: The first coordinate of the eigenvector associated to 20.5 is (approximately) 0. What are its two other coordinates?

Oral question: I want to check that the expansion formula for $\mathbb{V}(Z_1 + Z_2)$ holds where Z_i is the i^{th} coordinate of Z. How to do with  ?

True parameters: $\boldsymbol{\mu} = (1, 0, -1)$, and $\Sigma = \begin{pmatrix} 2 & 0.6 & -0.4 \\ 0.6 & 2 & 4 \\ -0.4 & 2 & 20 \end{pmatrix}$.

Exercise 4

The file "Exam_data.R" provides a function `add_noise`.

1. Write and run `nbr = 20`. Create two vectors `X` and `Y` containing `nbr` samples of the uniform distribution on $[0, 1]$. Create a `data.frame` called `df` containing the columns `X` and `Y` you have created.
2. Call the manual to understand the function `expand.grid`. Create a `data.frame` called `df2` containing two columns named `X` and `Y` but containing one rows for each (x_i, y_j) where $x_i \in$ `X` and $y_j \in$ `Y`.
3. Plot `df` and plot `df2`. Run `noisy_df = add_noise(df)` and `noisy_df2 = add_noise(df2)` and plot the result.
4. We want to distinguish between the two types of data. Run `cor.test(...)` on the four data frames, and comment the results.
5. Filter your four data frames under the condition that the x-value is smaller than the y-value, then re-plot and re-run the correlation tests on the four filtered data frames. Comment the results.

Oral question: How to compute a covariance matrix? What do you expect to see in the eight cases?

Oral question: How to compute the marginal distribution on the x-axis? If I run a Kolmogorov–Smirnov test against the uniform distribution for this marginal distribution, what should I expect in the eight cases?

Oral question: How to do if I want to plot `X` on the vertical axis and `Y` on the horizontal axis instead?

Exercise 5

The file "Exam_data.R" contains a function `f` and a function `draw_derivatives`.

1. Write and run `nbr = 75` and `rg = 4`. Create a vector `X` containing `nbr` samples of the uniform distribution on $[0, 20]$. Sort these samples, and create a `data.frame` called `df` containing the columns `X` and `Y = f(X)`.
2. Create a `data.frame` named `ddf` whose first column is `X`, second column is named `dY` and is full of `NA`, and third column is named `InterceptY` and is full of `NA`.
3. For each `k` between `1 + rg` and `nbr - rg`, construct `sub_df` to be the `data.frame` containing the rows of `df` from `k - rg` to `k + rg`. Compute the linear regression for `Y` against `X` for the data of this `sub_df`, and store the resulting coefficient linear coefficient in `ddf$dY[k]` and the constant coefficient in `ddf$InterceptY[k]`.
4. Plot `df`, and run `draw_derivatives(ddf, rg+2)`. Comment the result.
5. Some x-value `X[k]` is a *local minimum* of `f` if `dY[k-1]` is negative and `dY[k+1]` is positive. Find the local minima of `f`, and add them in red on the horizontal axis of the plot.

Oral question: How to find the local maxima?

Oral question: What do you expect as `rg` grows?

Oral question: How can I compute the second derivative of a function? What would be the problem?